SYSTEM ANALYSIS & DESIGN

Predictive Maintenance System

A system design document for machine failure prediction

PROJECT PROFILE	DETAIL
Application area	Industrial machine-failure prediction
Data source	AI4I 2020 Predictive Maintenance Dataset
Development stack	Python, pandas, scikit-learn / CatBoost
Planned application	FastAPI service and Streamlit dashboard
Deployment approach	Local implementation first; AWS deployment after validation

Prepared as a project design reference. The machine-learning model is still being evaluated; application and deployment components described in this document represent the planned implementation.

Document purpose

This document defines the requirements and proposed design for a predictive maintenance application. It describes the problem being addressed, the people who will use the system, the information it needs, and the way its main software components will work together.

The project uses the AI4I 2020 dataset to develop a binary classifier for machine failure. The intended application will let a user submit operating measurements and receive a model-generated failure estimate. The result is intended to support maintenance decisions; it is not an automatic machine-control command.

Project status

Data inspection, preparation, model experimentation, and cross-validation have been carried out in Python using Google Colab. Model selection is ongoing. The API, dashboard, prediction-history storage, and AWS deployment have not yet been implemented.

Contents

01 Project background and objectives

02 Stakeholders and system scope

03 Requirements specification

04 Feasibility and constraints

05 Proposed system architecture

06 Process flow and API design

07 UML design views

08 Data and machine-learning design

09 User interface design

10 Security, quality, and error handling

11 Verification and testing

12 Implementation and deployment plan

13 Assumptions and future development

01 Project background and objectives

1.1 Background

Unplanned equipment failures can interrupt production, increase maintenance expenses, and affect delivery schedules. Maintenance based only on fixed intervals may not reflect the actual condition of a machine. Predictive maintenance offers another approach: use operating data to estimate the likelihood of a failure and help maintenance staff decide when an inspection may be needed.

1.2 Problem definition

The project investigates whether machine operating measurements can be used to identify records associated with machine failure. The model receives six attributes: machine type, air temperature, process temperature, rotational speed, torque, and tool-wear time. The target field is **Machine failure**, where 1 indicates a recorded failure and 0 indicates no recorded failure.

1.3 Project objectives

- Prepare the dataset and identify features that can be used without target leakage.
- Compare classification algorithms using evaluation measures suitable for imbalanced data.
- Select a model based on validation and cross-validation performance.
- Provide a simple interface for submitting machine measurements and viewing predictions.
- Keep the model's preprocessing steps consistent between training and application use.
- Complete and test the application locally before allocating cloud resources.

1.4 Scope boundaries

Included in the first release	Not included in the first release
Single-record failure prediction	Automatic machine shutdown or control
Manual entry of six machine attributes	Direct PLC, SCADA, or sensor integration
Prediction probability and class	Remaining-useful-life estimation
Basic prediction history	Automated work-order management
Local API and dashboard	Fleet-wide optimization

02 Stakeholders and system scope

Stakeholder	Interest / responsibility
Machine operator	Enters or confirms operating measurements and checks the returned result.
Maintenance engineer	Reviews high-risk predictions and decides whether inspection is required.
Maintenance manager	Uses prediction history and summaries to support maintenance planning.
System administrator	Maintains application settings, user access, and the active model version.
ML developer	Trains, evaluates, versions, and prepares model updates.

2.1 System boundary

The initial system begins when a user enters machine measurements and ends when the prediction is displayed and, where enabled, recorded. Data acquisition from industrial equipment is outside the first release. The system does not make maintenance decisions on behalf of the engineer.

2.2 Operating assumptions

- Users have access to the dashboard through a supported web browser.
- The first version receives data manually or from prepared sample records.
- A selected and tested model artifact is available before the prediction service is released.
- Prediction results are reviewed alongside normal maintenance procedures.

03 Requirements specification

3.1 Functional requirements

ID	Requirement	Priority
FR-01	Accept machine type and five numeric operating measurements.	High
FR-02	Check required fields, data types, and acceptable input ranges.	High
FR-03	Apply the preprocessing pipeline saved with the selected model.	Critical
FR-04	Return a failure probability and a predicted class.	Critical
FR-05	Present the prediction in a clear format for the user.	High
FR-06	Record prediction time, inputs, result, and model version.	Medium
FR-07	Allow authorized users to review recent prediction records.	Medium
FR-08	Expose service health and active model information.	Medium

3.2 Non-functional requirements

Area	Requirement
Performance	A single prediction should normally return within a few seconds on the selected host.
Reliability	Invalid input or an unavailable model must produce an explicit error.
Usability	Labels and units should be visible beside each input field.
Maintainability	Separate the dashboard, API, validation, inference, and storage code.
Reproducibility	Record model version and the evaluation results used to approve it.
Security	Protect remote access and keep credentials outside the source code.
Portability	The application should run locally before it is packaged for deployment.

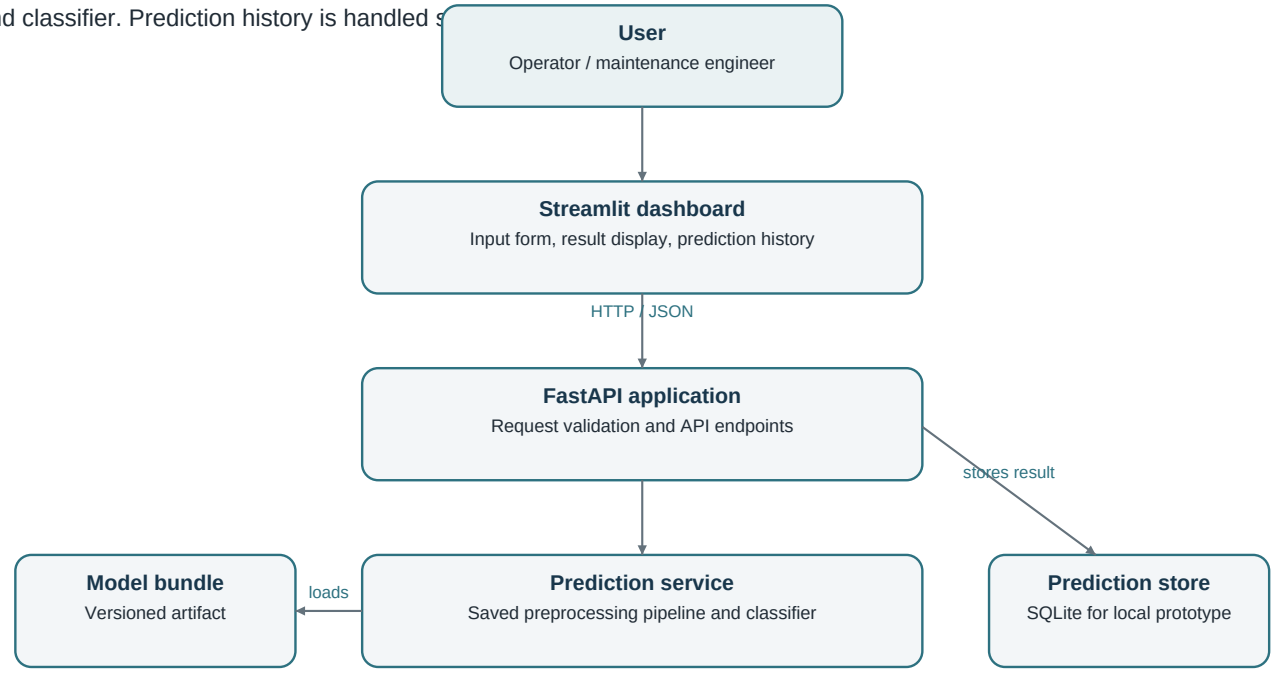
3.3 Business and data rules

- **UDI** and **Product ID** are identifiers and are not model input features.
- **TWF, HDF, PWF, OSF, and RNF** describe failure subtypes and are excluded from predictors to avoid target leakage.
- The model input consists of the six selected features only.
- The production pipeline must use the same feature names, order, and fitted transformations as the approved training pipeline.
- A high predicted risk is an indication for human review, not an instruction to stop equipment.

04 Feasibility and constraints

05 Proposed system architecture

The application is divided into a presentation layer, an API layer, and a prediction layer. The dashboard collects the machine readings, FastAPI validates each request, and the prediction service applies the saved preprocessing pipeline and classifier. Prediction history is handled s

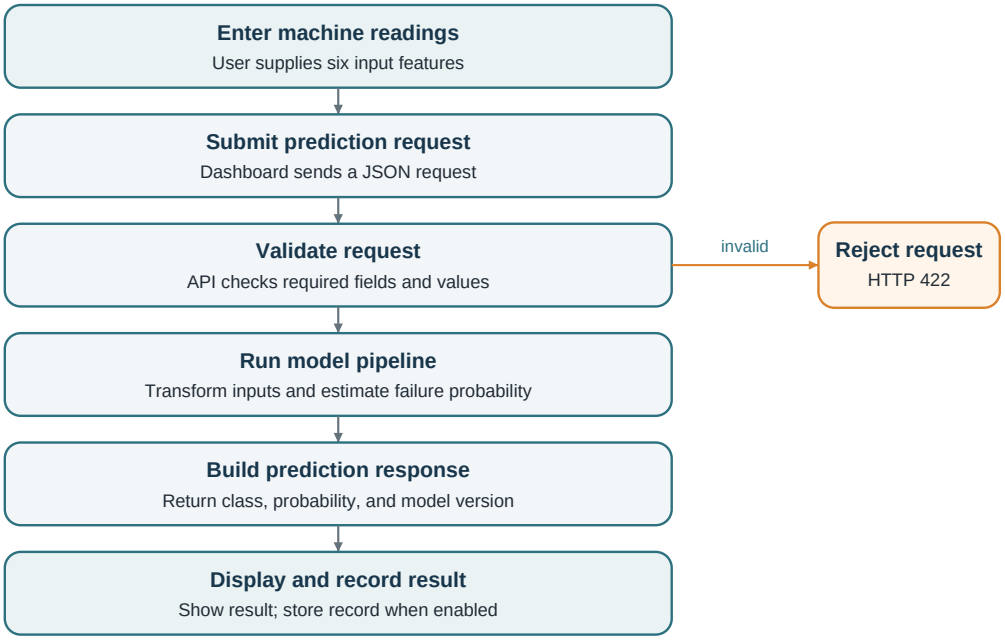


Component	Responsibility
Streamlit dashboard	Collects six inputs and presents prediction results and history.
FastAPI application	Validates requests and exposes the system endpoints.
Prediction service	Runs the fitted preprocessing pipeline and classifier.
Model bundle	Holds the approved model, transformer, and version metadata.
Prediction store	Keeps timestamped results for later review; SQLite is proposed locally.

06 Process flow and API design

6.1 Prediction process

The normal path continues only after the request passes validation. Invalid values are rejected with an HTTP 422 response; the model is not called for an invalid request.



6.2 API endpoints

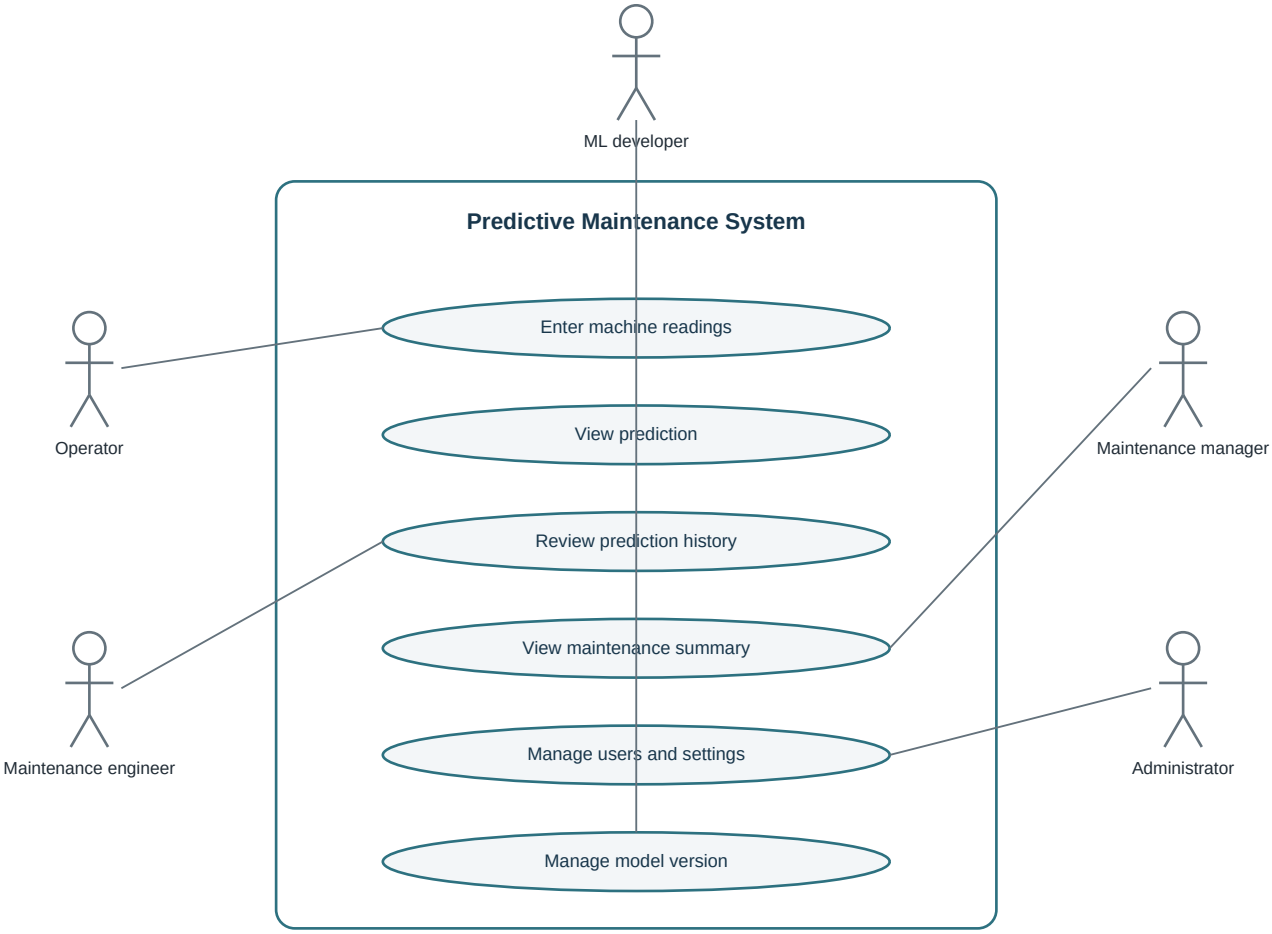
Method / endpoint	Purpose
GET /health	Checks API availability and model loading status.
POST /predict	Accepts one machine record and returns a prediction.
GET /predictions	Returns recent prediction records to authorized users.
GET /model-info	Returns active model version and approved evaluation summary.

The prediction request contains the machine type and five numeric measurements: air temperature, process temperature, rotational speed, torque, and tool wear.

07 UML design views

7.1 Use case diagram

The system boundary contains the functions provided by the application. Actors remain outside the boundary and are connected only to the functions relevant to their roles.

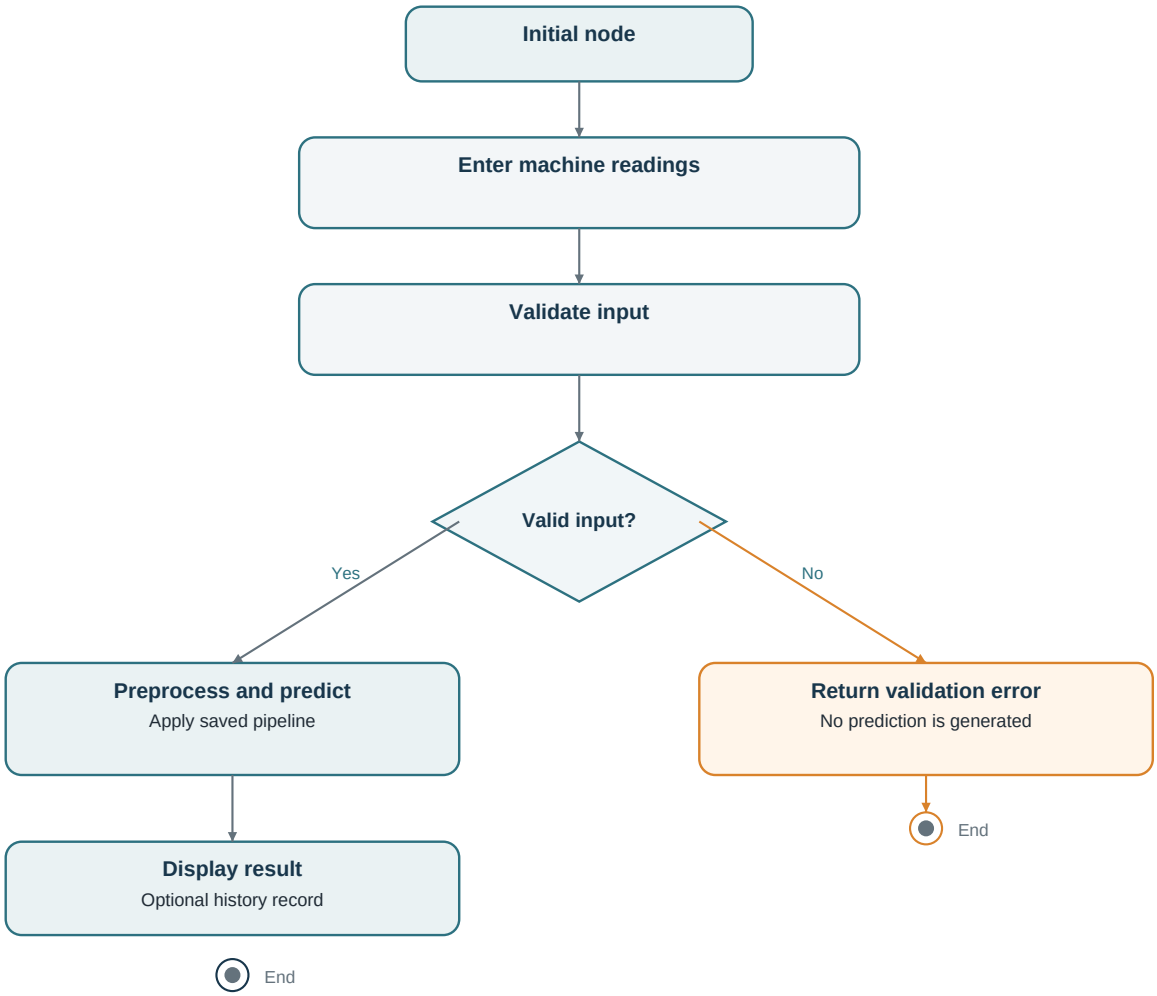


The operator enters readings and views predictions. The maintenance engineer reviews high-risk results and history. The manager reviews summaries, the administrator manages application settings, and the ML developer manages model versions.

07 UML design views — activity

7.2 Activity diagram

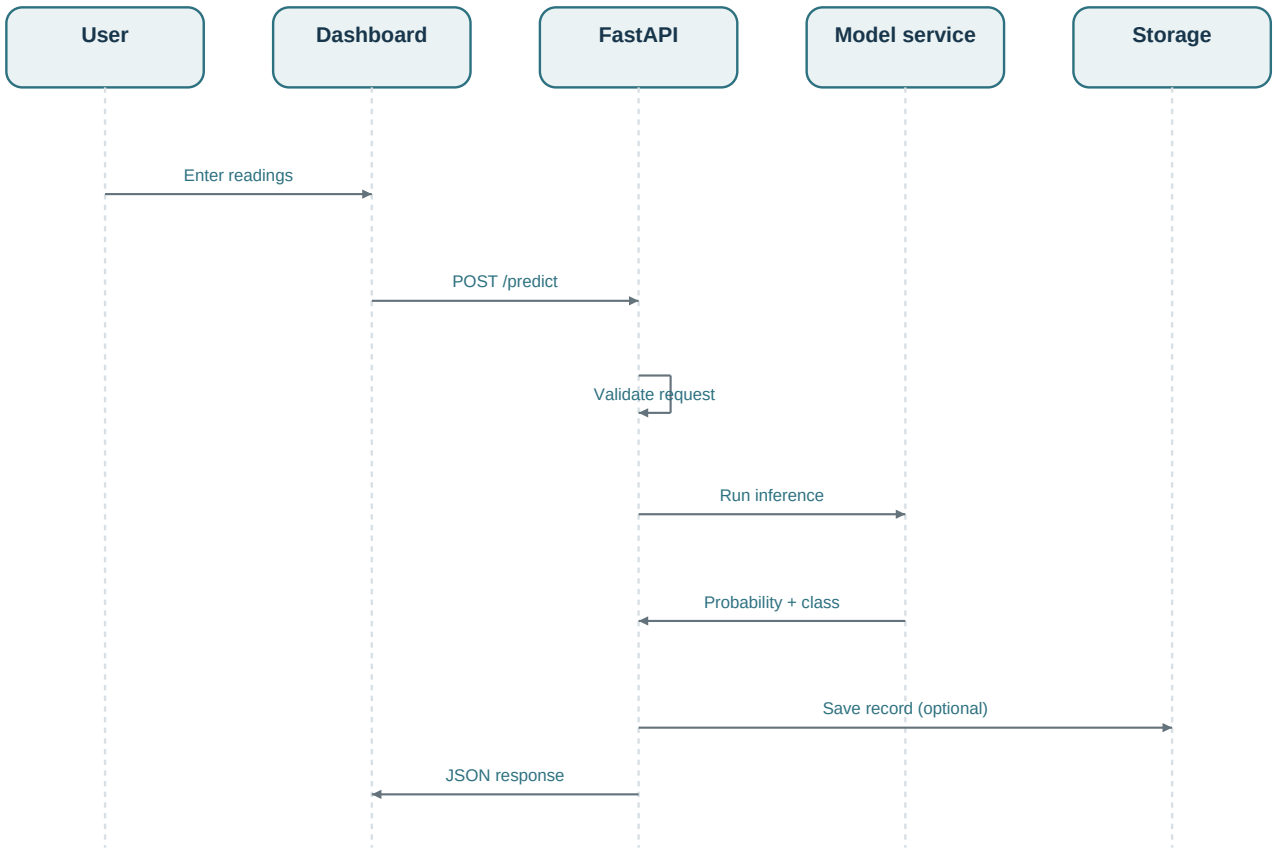
The activity diagram shows the validation decision and the two possible outcomes. An invalid request ends after the validation message; a valid request continues through prediction and result display.



07 UML design views — sequence

7.3 Sequence diagram

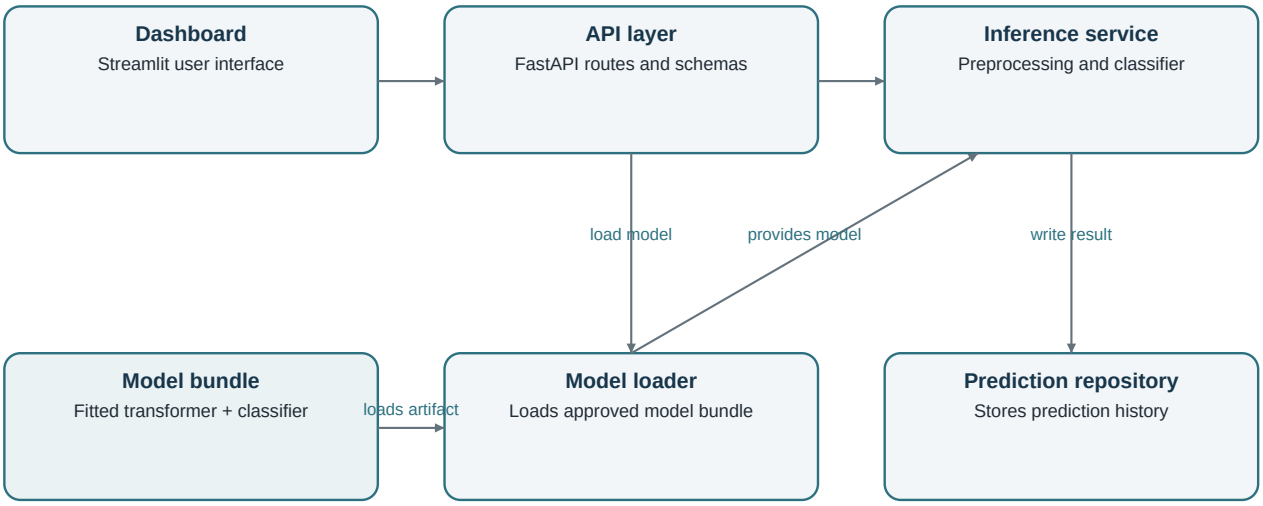
The API validates the incoming request before invoking the model service. The service returns a probability and predicted class. The API may store the result before returning the response to the dashboard.



07 UML design views — components

7.4 Component diagram

The component diagram separates presentation, request handling, inference, model loading, and prediction storage. The model loader reads the approved model bundle; the inference service uses the loaded pipeline to generate predictions.

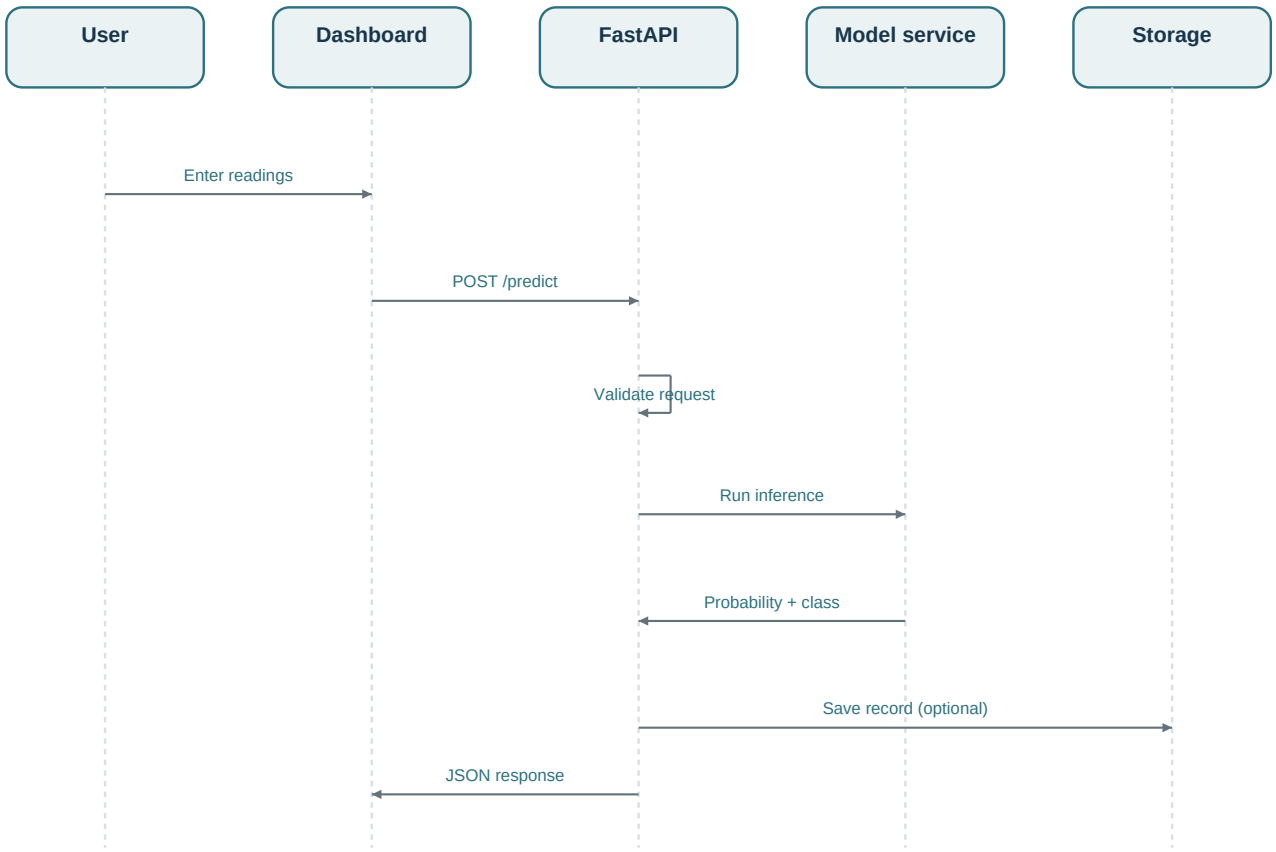


Module	Purpose
Dashboard	User-facing input and result screens.
API layer	Request validation and endpoint handling.
Inference service	Preprocessing and prediction.
Model loader / bundle	Loads the approved transformer and classifier.
Prediction repository	Stores prediction history.

07 UML design views — continued

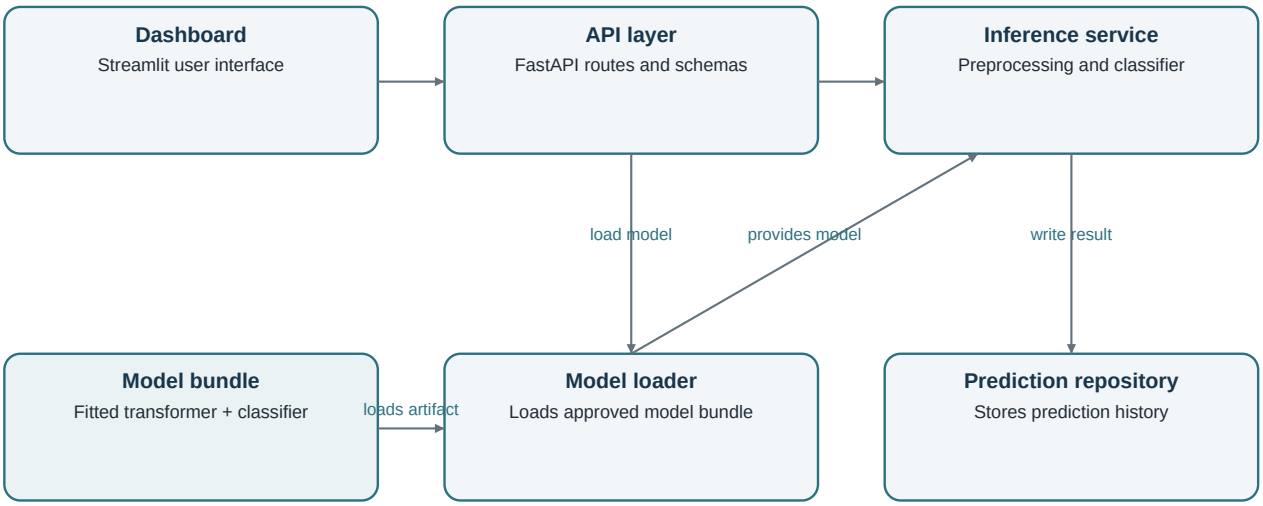
7.3 Sequence diagram

The sequence diagram shows the order of messages during a prediction request. Storage is optional and is used only when prediction history is enabled.



7.4 Component diagram

The component diagram separates the user interface, API, inference service, model loading, model artifact, and prediction repository.


Dashboard
Streamlit user interface
API layer
FastAPI routes and schemas
Inference service
Preprocessing and classifier
load model
provides model
write result
Model bundle
Fitted transformer + classifier
loads artifact
Model loader
Loads approved model bundle
Prediction repository
Stores prediction history

08 Data and machine-learning design

8.1 Model input fields

Field	Data type	Description
Type	Categorical	Machine category (L, M, or H in the dataset).
Air temperature [K]	Numeric	Ambient air temperature in Kelvin.
Process temperature [K]	Numeric	Machine process temperature in Kelvin.
Rotational speed [rpm]	Numeric	Rotational speed in revolutions per minute.
Torque [Nm]	Numeric	Operating torque in Newton-metres.
Tool wear [min]	Numeric	Accumulated tool-wear time in minutes.

8.2 Preprocessing approach

- Numeric columns may use median imputation and scaling, depending on the selected algorithm.
- The categorical Type field uses most-frequent imputation and one-hot encoding.
- Preprocessing is fitted within each training fold during cross-validation to avoid information leakage.
- The final fitted preprocessing object must be packaged with the classifier.

8.3 Model development and evaluation

Several classification approaches have been explored, including Logistic Regression, Random Forest, Extra Trees, HistGradientBoosting, XGBoost, and CatBoost. The model has not yet been finalized. The selection process should consider PR-AUC, recall, precision, F1 or F2, ROC-AUC, and the confusion matrix.

The project uses stratified training, validation, and test subsets. Cross-validation is performed on the training portion. The validation data is used for model comparison and threshold selection. The test set remains untouched until the final model and threshold have been chosen.

8.4 Threshold selection

The probability threshold controls the trade-off between missed failures and false alarms. A lower threshold may identify more failures but can increase unnecessary inspections. The final threshold should reflect the maintenance team's tolerance for these two types of error.

8.5 Prediction history fields

Field	Purpose
prediction_id	Unique identifier for the prediction record.
created_at	Date and time when the prediction was generated.
machine_type	Submitted machine category.
measurements	The submitted temperature, speed, torque, and tool-wear values.
failure_probability	Estimated probability for the failure class.
predicted_failure	Predicted binary class.
model_version	Version of the model used for the prediction.

09 User interface design

Screen	Content
Prediction	Form for the six features, field validation, Predict action, probability, predicted class, and risk label.
History	Recent predictions with timestamp, result, probability, and model version.
Overview	Counts of predictions and a basic risk distribution, when enough records are available.
Model information	Active model version, training date, dataset reference, and approved evaluation metrics.

The result screen should show the predicted class and probability together. It should explain that the probability is a model estimate, not a guarantee. Any Low, Medium, or High risk bands must be agreed and tested; the dataset does not define universal operational thresholds.

10 Security, quality, and error handling

Issue	Design response
Invalid input	Reject missing or incorrectly typed values and return a useful validation message.
Model unavailable	Return a service error and record the failure; do not generate a substitute prediction.
Unauthorized access	Require authentication for protected history and administration functions.
Credentials	Keep secrets in environment variables or a managed secret store.
Network traffic	Use HTTPS for remote deployment.
Audit trail	Record model version and prediction metadata according to a defined retention policy.
Model drift	Monitor input distributions and confirmed outcomes before deciding whether retraining is needed.

11 Verification and testing

Test level	Examples
Unit tests	Input validation, feature ordering, preprocessing consistency, model loading, and risk-label mapping.
API tests	Valid request, missing field, incorrect type, invalid category, and unavailable model.
Integration tests	Dashboard request reaches the API and the returned values are displayed correctly.
Model evaluation	Final evaluation on untouched test data using PR-AUC, ROC-AUC, precision, recall, F1/F2, and confusion matrix.
Usability tests	A user can enter measurements and understand the returned result.
Deployment tests	Service startup, health endpoint, environment settings, logging, and access restrictions.

Acceptance conditions

- Valid input returns the documented response fields.
- Invalid input is rejected with an understandable message.
- Inference uses the same feature definitions and fitted preprocessing as the approved model.
- Final model metrics are reported from the reserved test set after model selection is complete.
- The interface clearly presents predictions as advisory information.

12 Implementation and deployment plan

Stage	Work package	Current status
1	Dataset inspection, cleaning, leakage review, and stratified split.	Completed
2	Algorithm comparison, parameter tuning, and cross-validation.	In progress
3	Choose final model and threshold; evaluate once on the test set.	Pending
4	Package preprocessing and classifier with version metadata.	Pending
5	Build FastAPI endpoints and request validation.	Planned
6	Build Streamlit prediction and history screens.	Planned
7	Run local end-to-end and acceptance tests.	Planned
8	Deploy to AWS after local verification.	Future stage

12.1 Cloud deployment considerations

AWS deployment should take place only after the local API and dashboard work correctly. A simple hosting option should be compared against expected usage and cost. Amazon S3 may be used for versioned model artifacts; a small compute service can host the API. A managed database or SageMaker should be introduced only if the project requirements justify the additional cost and complexity.

12.2 Release checks

- Confirm the final model, threshold, and test results.
- Confirm the saved artifact loads in a clean local environment.
- Test the API and dashboard together.
- Configure environment variables, access controls, and logging.
- Estimate cloud costs and stop unused billable resources.

13 Assumptions and future development

13.1 Assumptions

- The first release relies on manually entered or sample machine readings.
- AI4I 2020 is the initial dataset; real equipment data may have different patterns.
- The application supports maintenance decisions but does not control equipment.
- Model selection and the operating threshold remain open until validation is complete.

13.2 Possible extensions

- Connect industrial sensors or an IoT data gateway.
- Support batch predictions and multiple machines.
- Link confirmed failures and maintenance outcomes to prediction history.
- Monitor model drift and schedule controlled retraining.
- Add feature explanations to help engineers interpret individual predictions.
- Evaluate the system using real industrial data before operational use.

Closing note

The proposed design provides a practical route from the current machine-learning experiments to a working predictive maintenance application. The separation between interface, API, inference, and storage keeps the system manageable and makes later changes easier. The next development priority is to complete model selection and final evaluation, then package the approved pipeline before building the application around it.

This document should be updated as implementation decisions are confirmed, particularly the selected model, threshold, storage approach, and deployment platform.